

React & Web Development Basics

JavaScript • TypeScript • React • Next.js • npm • APIs • Git • Build & Deployment

Complete Revision & Interview Notes



Contents

1. [Node.js](#)
2. [npm](#)
3. [npx](#)
4. [package.json & package-lock.json](#)
5. [dependencies vs devDependencies](#)
6. [Semantic Versioning](#)
7. [npm Scripts](#)
8. [Shell Operators](#)
9. [JSON.stringify & JSON.parse](#)
10. [ESLint & Prettier](#)
11. [Jest](#)
12. [TypeScript Basics](#)
13. [JavaScript Basics](#)
14. [React Basics](#)
15. [State & useState](#)
16. [useEffect](#)
17. [Context API](#)

18. [Async/Await & Promises](#)
19. [Fetch & APIs](#)
20. [HTTP Basics](#)
21. [Next.js Basics](#)
22. [Server vs Client Components](#)
23. [Supabase Basics](#)
24. [Environment Variables](#)
25. [Git Basics](#)
26. [Build Process](#)
27. [Interview Quick Revision](#)

1. Node.js

Node.js is a JavaScript runtime that allows JavaScript to run outside the browser.

Normally:

JavaScript → Browser → Chrome / Edge / Firefox

With Node.js:

JavaScript → Node.js → Operating System

Why do we need Node.js?

- Run JavaScript on the server.
- Run development tools.
- Run npm commands.
- Build React/Next.js applications.
- Create backend applications.

Check Node version

```
node -v
```

Example:

v24.18.0

Remember: Node.js is the runtime. npm is the package manager that normally comes with Node.js.

2. npm

npm stands for **Node Package Manager**.

It is used to:

- Install packages.
- Remove packages.
- Update packages.
- Run project scripts.
- Manage project dependencies.

Install a package

```
npm install react
```

Short version:

```
npm i react
```

Install development dependency

```
npm install -D eslint
```

or:

```
npm install --save-dev eslint
```

Remove package

```
npm uninstall react
```

Install everything from package.json

```
npm install
```

Important: `npm install` does not mean "install Node.js". It installs the project's packages.

3. npx

npx is commonly used to execute package binaries.

Example:

```
npx create-vite@latest
```

Another example:

```
npx tsc --init
```

It can use a locally installed package binary, or obtain a package when necessary.

Command	Purpose
npm install	Install packages
npm run	Run package.json scripts
npx	Execute package binaries



Think: **npm installs, npx executes.**

4. package.json & package-lock.json

package.json

This is the main configuration file of a Node/React project.

```
{
  "name": "my-app",
  "version": "1.0.0",
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "eslint ."
  },
  "dependencies": {
    "react": "^19.0.0"
  }
}
```

It contains information such as:

- Project name
- Version
- Scripts
- Dependencies

- Development dependencies

package-lock.json

The lock file records the resolved dependency versions and dependency tree so installs can be reproduced more consistently.

package.json: What the project declares it needs.

package-lock.json: The resolved dependency tree npm recorded.

5. dependencies vs devDependencies

dependencies

Packages required by the application at runtime or by the deployed application environment.

```
npm install react
```

Goes into:

```
"dependencies": {  
  "react": "..."  
}
```

devDependencies

Packages primarily used during development/building/testing.

```
npm install -D eslint
```

Goes into:

```
"devDependencies": {  
  "eslint": "..."  
}
```

dependencies

devDependencies

Runtime/application packages	Development/tooling packages
React	ESLint
Some runtime libraries	Prettier
Backend libraries	Jest / testing tools

Do not decide purely from the package name. Whether a package belongs in dependencies or devDependencies depends on how the project uses it and how the deployment/build environment installs packages.

6. Semantic Versioning

A version commonly looks like:

1.2.3

Meaning:

MAJOR . MINOR . PATCH

Part	Meaning
Major	Breaking changes
Minor	Backward-compatible features
Patch	Bug fixes

^

```
"react": "^19.0.0"
```

For normal 1.x-or-later versions, ^ allows compatible minor/patch updates below the next major version.

~

```
"package": "~1.2.3"
```

Generally allows patch updates below 1.3.0.

Important: package-lock.json can cause npm to install the exact resolved version recorded there, even when package.json contains a range.

7. npm Scripts

Scripts are defined inside `package.json` .

```
"scripts": {  
  "dev": "next dev",  
  "build": "next build",  
  "start": "next start",  
  "lint": "eslint ."  
}
```

Run script

```
npm run dev
```

Special scripts

You can run:

```
npm start  
npm test
```

without `npm run` because npm treats some lifecycle names specially.

npm run build

Runs whatever command the project defines under the `build` script.

For example:

```
"build": "next build"
```

means:

```
npm run build
```

executes:

```
next build
```

prebuild

```
"prebuild": "node scripts/generate.js",  
"build": "next build"
```

When you run:

```
npm run build
```

npm automatically runs:

```
prebuild → build
```

Remember: `prebuild` is a lifecycle hook that runs before the `build` script.

8. Shell Operators

&& — AND

```
npm install && npm run build
```

Means:

Run command 2 only if command 1 succeeds.

> — Redirect output

```
npm run build > build.log
```

Writes standard output to a file.

| — Pipe

```
node -p "process.version" | findstr 24
```

The output of the first command becomes input to the second command.

Operator	Meaning
&&	Run next command if previous succeeds
>	Redirect output
	Pipe output into another command

9. JSON.stringify() & JSON.parse()

JSON.stringify()

Converts a JavaScript value into a JSON string.

```
const user = {  
  name: "Munna",  
  age: 30  
};  
  
const json = JSON.stringify(user);  
  
console.log(json);
```

Result:

```
{"name": "Munna", "age": 30}
```

JSON.parse()

Converts a JSON string into a JavaScript value.

```
const json = '{"name":"Munna","age":30}';  
  
const user = JSON.parse(json);  
  
console.log(user.name);
```

JavaScript Object

↓ **JSON.stringify()**

JSON String

↓ **JSON.parse()**

JavaScript Object

Easy memory trick:

stringify = object → string

parse = string → object

10. ESLint & Prettier

ESLint

ESLint analyzes code and identifies problems, suspicious patterns, and style-rule violations.

```
npm run lint
```

Example problem:

```
const unusedVariable = 10;
```

Depending on configuration, ESLint may warn about or report the unused variable.

Prettier

Prettier is primarily a code formatter.

Example:

```
const user={name:"Munna",age:30}
```

Prettier can format it as:

```
const user = {  
  name: "Munna",  
  age: 30,  
};
```

Tool	Main purpose
ESLint	Find code-quality/rule problems
Prettier	Format code consistently

11. Jest

Jest is a JavaScript testing framework.

```
test("adds numbers", () => {  
  expect(2 + 3).toBe(5);  
});
```

The test checks whether:

```
2 + 3 === 5
```

Why testing?

- Find bugs early.
- Prevent regressions.
- Verify functions.
- Increase confidence when changing code.

12. TypeScript Basics

TypeScript is JavaScript with a static type system.

Variable type

```
let name: string = "Munna";

let age: number = 30;

let isAdmin: boolean = false;
```

Interface

```
interface User {
  name: string;
  age: number;
  email: string;
}
```

Use it:

```
const user: User = {
  name: "Munna",
```

```
    age: 30,  
    email: "munna@example.com"  
  };
```

Function return type

```
function add(a: number, b: number): number {  
    return a + b;  
}
```

Promise return type

```
async function getUsers(): Promise {  
    // ...  
}
```

This means the function eventually resolves to an array of User objects.

Important: TypeScript types are mainly compile-time information. They do not automatically validate values at runtime.

13. JavaScript Basics

const

```
const name = "Munna";
```

The variable binding cannot be reassigned.

let

```
let count = 0;
```

```
count = 1;
```

The variable can be reassigned.

Arrow function

```
const add = (a, b) => {  
  return a + b;  
};
```

Implicit return

```
const add = (a, b) => a + b;
```

These are equivalent:

```
const double = x => x * 2;
```

```
const double = x => {  
  return x * 2;  
};
```

map()

```
const numbers = [1, 2, 3];
```

```
const result = numbers.map(number => number * 2);
```

Result:

```
[2, 4, 6]
```

If you use curly braces in an arrow function, you need an explicit `return` if you want to return a value.

```
// WRONG for rendering a value
numbers.map(number => {
  number * 2;
});
```

```
// CORRECT
numbers.map(number => {
  return number * 2;
});
```

```
// Also CORRECT
numbers.map(number => number * 2);
```

14. React Basics

React is a JavaScript library for building user interfaces.

Component

```
function Welcome() {  
  return
```

Hello

```
;  
}
```

A React component is generally a function that returns UI.

JSX

```
const element =
```

Hello Munna

```
;
```

JSX lets you write HTML-like syntax inside JavaScript/TypeScript.

Props

```
function User({ name }) {  
  return
```

Hello {name}

```
;  
}
```

Use:

```
<User name="Munna" />
```

Props are inputs passed from a parent component to a child component.

15. State & useState

State represents data that can change during a component's lifetime and whose changes can cause a re-render.

```
const [count, setCount] = useState(0);
```

There are two variables:

Part	Meaning
count	Current state value
setCount	Function used to update state
0	Initial value

Example

```
const [count, setCount] = useState(0);
```

```
setCount(count + 1)>  
Count: {count}
```

Why not directly modify state?

```
// Don't do this  
count = count + 1;
```

Instead:

```
setCount(count + 1);
```

Calling the state setter tells React that state changed and allows React to schedule a re-render.

16. useEffect

`useEffect` is used for side effects in client components.

Examples of side effects:

- Fetching data.
- Reading localStorage.
- Updating document title.
- Subscribing to events.
- Timers.

```
useEffect(() => {  
  console.log("Component mounted");  
}, []);
```

The empty dependency array means the effect does not depend on changing values and normally runs after the initial render. In React development Strict Mode, effects may be intentionally re-run to help detect issues.

Dependencies

```
useEffect(() => {  
  console.log(count);  
}, [count]);
```

This effect runs when `count` changes.

17. Context API

Context allows data to be made available to components deeper in the component tree without passing props through every level.

Create context

```
interface ThemeContextType {  
  theme: "light" | "dark";  
  toggleTheme: () => void;  
}  
  
const ThemeContext =  
  createContext<ThemeContextType | undefined>(undefined);
```

Provider

```
<ThemeContext.Provider  
  value={{ theme, toggleTheme }}  
>  
  {children}  
</ThemeContext.Provider>
```

The Provider makes `theme` and `toggleTheme` available to descendants.

useContext

```
const context = useContext(ThemeContext);
```

If no Provider is above the component and the default is undefined, context will be undefined.

Custom hook

```
export function useTheme() {  
  const context = useContext(ThemeContext);  
  
  if (!context) {  
    throw new Error(  
      "useTheme must be used inside ThemeProvider"  
    );  
  }  
  
  return context;  
}
```

Why throw an error? It gives a clear developer error instead of allowing the application to fail later because context is undefined.

18. Async/Await & Promises

An asynchronous operation may finish later.

Promise

A Promise represents a future result.

async

```
async function getData() {  
  return "Hello";  
}
```

An async function always returns a Promise.

await

```
const result = await getData();
```

`await` waits for the Promise result inside an async function.

Explicit TypeScript type

```
async function getUsers(): Promise<User[]> {  
  const users = await getData();  
  return users;  
}
```

Meaning:

getUsers()
↓ **Promise**
↓ **User[]**

try/catch

```
try {  
  const data = await getData();  
} catch (error) {  
  console.error(error);  
}
```

19. Fetch & APIs

Basic fetch

```
const res = await fetch("/api/users");
```

`res` is a **Response object**.

It is not automatically the JSON data.

Read JSON

```
const data = await res.json();
```

Now:

fetch()
↓ **Response object**
↓ **res.json()**
JavaScript data

Example

```
const res = await fetch("/api/users");

if (!res.ok) {
  throw new Error("Request failed");
}

const users = await res.json();
```

POST request

```
const res = await fetch("/api/users", {
  method: "POST",
  headers: {
    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    name: "Munna",
    age: 30
  })
});
```

Remember: `JSON.stringify()` is commonly used to convert JavaScript data into the JSON string sent in an HTTP request body.

20. HTTP Basics

HTTP is the protocol commonly used for communication between clients and servers.

Method	Common purpose
GET	Retrieve data
POST	Create/send data
PUT	Replace/update a resource
PATCH	Partially update a resource
DELETE	Delete a resource

Status codes

Status	Meaning
200	OK
201	Created
400	Bad Request

401	Unauthenticated / authentication required
403	Forbidden
404	Not Found
500	Internal Server Error

21. Next.js Basics

Next.js is a React framework.

It provides features such as:

- Routing
- Server Components
- Client Components
- Server-side functionality
- API route handlers
- Build optimization
- Image optimization
- Deployment support

App Router

A common structure:

```
app/  
├─ page.tsx  
├─ layout.tsx  
├─ about/  
│   └─ page.tsx  
└─ dashboard/  
    └─ page.tsx
```

Routes are based on folders/files.

app/about/page.tsx

↓ **/about**

layout.tsx

Defines shared UI/layout around pages.

page.tsx

Defines the UI for a route.

22. Server Components vs Client Components

Server Components

In the Next.js App Router, components are Server Components by default.

They are useful for:

- Server-side data fetching.
- Accessing server-side resources.
- Keeping server-only logic on the server.
- Reducing client-side JavaScript.

Client Components

Add:

```
"use client";
```

when the module needs client-side capabilities such as:

- useState
- useEffect
- event handlers
- browser APIs
- interactive UI

"use server"

"use server" is associated with Server Actions/server-side functions. It is not simply the opposite of "use client" and is not a general marker saying "this whole component is a Server Component."

Easy memory:

Default Next.js App Router component → Server Component

Need browser interaction/hooks → "use client"

Server Action/function → "use server"

23. Supabase Basics

Supabase provides backend services including a PostgreSQL database and authentication.

Query example

```
const { data, error } = await supabase
  .from("profiles")
  .select("*")
  .eq("id", user.id)
  .single();
```

data

Contains the returned database record when successful.

error

Contains an error if the query failed.

single()

`single()` expects exactly one row.

If zero or multiple rows are returned, it results in an error.

maybeSingle()

`maybeSingle()` allows zero rows and returns `null` for that case, but still expects no more than one row.

Example

```
const { data, error } = await supabase
  .from("profiles")
  .select("*")
  .eq("id", user.id)
  .single();

if (error) {
  throw error;
}

return data;
```

If your function is supposed to return the profile object, return `data` , not the entire `{ data, error }` response object.

24. Environment Variables

Environment variables store configuration outside source code.

Example:

```
NEXT_PUBLIC_SUPABASE_URL=...  
NEXT_PUBLIC_SUPABASE_ANON_KEY=...
```

Access in Next.js

```
process.env.NEXT_PUBLIC_SUPABASE_URL
```

Non-null assertion

```
process.env.NEXT_PUBLIC_SUPABASE_URL!
```

The `!` tells TypeScript:

"Assume this value is not undefined."

It does not perform runtime validation.

Safer validation

```
const url = process.env.NEXT_PUBLIC_SUPABASE_URL;

if (!url) {
  throw new Error("Missing Supabase URL");
}
```

Important: In Next.js, environment variables prefixed with `NEXT_PUBLIC_` are intended to be available to client-side code. Never put secrets in public environment variables.

25. Git Basics

Git is a version control system.

Initialize repository

```
git init
```

Check status

```
git status
```

Add files

```
git add .
```

Commit

```
git commit -m "Initial commit"
```

Push

```
git push
```

Pull

```
git pull
```

Clone

```
git clone <repository-url>
```

Branches

```
git branch
```

```
git switch -c feature/login
```

Working Directory

↓ **git add**

Staging Area

↓ **git commit**

Repository

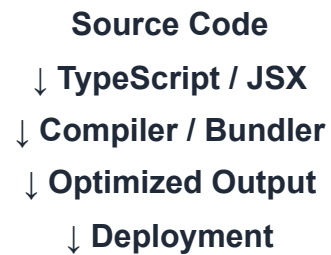
↓ **git push**

Remote Repository

26. Build Process

During development, tools make it convenient to write and test code.

For production, the project is usually built/optimized.



Typical Next.js commands

```
npm run dev
```

Starts the development server.

```
npm run build
```

Creates the production build according to the Next.js project configuration.

```
npm start
```

Runs the project's configured start command, commonly the production server after a successful build.

The exact behavior of `npm run build` and `npm start` depends on the scripts defined in `package.json`.

27. react-scripts

`react-scripts` is associated with Create React App (CRA).

It provided common scripts/tooling such as:

```
react-scripts start  
react-scripts build  
react-scripts test
```

It is not a requirement for React itself, and Next.js uses its own build tooling rather than `react-scripts`.

28. react-dotenv

`react-dotenv` is a third-party package used in some React setups for environment-variable handling.

Do not confuse third-party packages with built-in React features. Environment-variable handling depends on the build framework/toolchain you are using.

29. node -p

`node -p` evaluates a JavaScript expression and prints the result.

```
node -p "process.version"
```

Example output:

```
v24.18.0
```

You can also inspect:

```
node -p "process.platform"
```

30. require()

`require()` is commonly used to load modules in the CommonJS module system.

```
const fs = require("fs");
```

In modern JavaScript/TypeScript projects using ES Modules, you will commonly see:

```
import fs from "fs";
```

CommonJS: `require()`

ES Modules: `import/export`

31. Complete API Request Flow

Understand this flow very well because it appears constantly in frontend development.



Example

```
async function getUsers() {  
  
    const response = await fetch("/api/users");  
  
    if (!response.ok) {  
        throw new Error("Failed to fetch users");  
    }  
}
```

```
}  
  
const users = await response.json();  
  
return users;  
}
```

32. fetch() and cache: "no-store"

```
const response = await fetch("/api/users", {  
  cache: "no-store"  
});
```

In Next.js, this option tells the framework not to use cached data for that fetch and to request fresh data rather than serving a cached fetch result.

Caching behavior can depend on the Next.js version and the surrounding rendering/data-fetching setup, so always check the framework's current rules when debugging caching.

33. localStorage

`localStorage` allows browser-side key/value storage.

Save

```
localStorage.setItem("theme", "dark");
```

Read

```
const theme = localStorage.getItem("theme");
```

The return type is:

```
string | null
```

Remove

```
localStorage.removeItem("theme");
```

JSON storage

```
localStorage.setItem(  
  "user",  
  JSON.stringify(user)  
);
```

Read it back:

```
const user = JSON.parse(  
  localStorage.getItem("user")  
);
```

34. TypeScript Type Assertion

Example:

```
const savedTheme =  
  localStorage.getItem("theme") as Theme | null;
```

`as` is a type assertion.

You are telling TypeScript how you want the value treated at compile time.

A type assertion does NOT validate the value at runtime.

For example:

```
const value = "something" as Theme;
```

TypeScript may accept the assertion, but the runtime value is still just a string.

35. Cookies & Supabase SSR

In a server-side context, cookies can be used to carry session/authentication information.

A Next.js example may look like:

```
const cookieStore = await cookies();
```

When processing cookies:

```
cookies.forEach(  
  ({ name, value, options }) => {  
    cookieStore.set(  
      name,  
      value,  
      options  
    );  
  }  
);
```

This:

```
{ name, value, options }
```

is object destructuring.

It is equivalent to:

```
cookie => {  
  const name = cookie.name;  
  const value = cookie.value;  
  const options = cookie.options;  
}
```

36. SWR & mutate()

SWR is a React data-fetching library.

Example:

```
const {  
  data: session,  
  isLoading: loading,  
  mutate  
} = useSWRHandler("/api/session", fetcher);
```

The function named `mutate` is coming from the hook's return value.

If the custom hook wraps SWR, it may ultimately come from SWR's own API.

What does mutate() do?

It can be used to update/revalidate the cached data associated with the current key.

```
mutate();
```

Often this means:

Existing cached data

↓ **Revalidate / update**

↓ **Fresh data**

↓ **Component updates**

37. Callback Functions

A callback is a function passed to another function.

```
function greet(name, callback) {  
  console.log("Hello " + name);  
  callback();  
}
```

```
greet("Munna", () => {  
  console.log("Done");  
});
```

forEach

```
items.forEach(item => {  
  console.log(item);  
});
```

The function passed to `forEach` is a callback.

map

```
items.map(item => item.name);
```

The callback produces a value that becomes an element in the new array.

38. Props & TypeScript

Suppose a component expects:

```
interface NavbarProps {  
  full_name: string | null;  
}  
  
function Navbar({ full_name }: NavbarProps) {  
  return  
  {full_name}  
;  
}
```

Then:

```
<Navbar full_name={profile.full_name} />
```

is correct.

But:

```
<Navbar userName={profile.full_name} />
```

is incorrect if the component expects `full_name` .



Prop names must match the component's expected props.

39. Basic SQL Concepts

Table

A table stores rows and columns.

Primary Key

Uniquely identifies a row.

```
PRIMARY KEY (id)
```

Foreign Key

Creates a relationship to another table.

```
id UUID REFERENCES auth.users(id)
```

ON DELETE CASCADE

```
ON DELETE CASCADE
```

Means related child rows can automatically be deleted when the referenced parent row is deleted.

RLS

Row Level Security controls which rows users can access based on database policies.

```
ALTER TABLE profiles  
ENABLE ROW LEVEL SECURITY;
```

40. How Everything Connects



This is the big picture you should understand rather than memorizing individual commands.

41. Interview Quick Revision

Q: What is Node.js?

Node.js is a JavaScript runtime that allows JavaScript to run outside the browser.

Q: What is npm?

npm is the Node package manager used to install/manage packages and run project scripts.

Q: What is npx?

npx is used to execute package binaries, commonly using a local package binary or obtaining the package when necessary.

Q: Difference between package.json and package-lock.json?

package.json declares project dependencies and scripts; package-lock.json records npm's resolved dependency tree and versions.

Q: What is React?

React is a JavaScript library for building user interfaces using components.

Q: What is JSX?

JSX is syntax that allows HTML-like UI markup to be written inside JavaScript/TypeScript.

Q: What are props?

Props are inputs passed from a parent component to a child component.

Q: What is state?

State is component data that can change and trigger a re-render when updated.

Q: What is useState?

useState is a React Hook used to add state to a function component.

Q: What is useEffect?

useEffect is a React Hook used to synchronize a component with external systems and perform side effects.

Q: What is Context API?

Context allows data to be shared with descendant components without passing props through every intermediate component.

Q: What does async mean?

An async function always returns a Promise.

Q: What does await do?

`await` waits for a Promise to settle inside an `async` function and gives the fulfilled value when successful.

Q: What does `fetch()` return?

`fetch()` returns a Promise that resolves to a Response object.

Q: What does `response.json()` do?

It reads the response body and parses it as JSON.

Q: What is `JSON.stringify()`?

It converts a JavaScript value into a JSON string.

Q: What is `JSON.parse()`?

It converts a JSON string into a JavaScript value.

Q: What is TypeScript?

TypeScript is a superset of JavaScript that adds static typing and other language features.

Q: What is an interface?

An interface describes the expected shape of an object in TypeScript.

Q: What does "use client" mean in Next.js?

It marks a module as a Client Component boundary so client-side features such as state, effects, event handlers, and

browser APIs can be used.

Q: Are Next.js components Server Components by default?

In the App Router, yes, components are Server Components by default.

Q: What is Git?

Git is a distributed version control system used to track changes in source code.

Q: What does git add do?

It stages changes for the next commit.

Q: What does git commit do?

It records the staged changes in the local Git repository.

Q: What does git push do?

It sends local commits to a remote repository.

42. One-Page Command Cheat Sheet

Command	Purpose
<code>node -v</code>	Check Node version
<code>npm -v</code>	Check npm version
<code>npm install</code>	Install dependencies
<code>npm install package</code>	Install package
<code>npm install -D package</code>	Install dev dependency
<code>npm uninstall package</code>	Remove package
<code>npm run dev</code>	Run dev script
<code>npm run build</code>	Run build script
<code>npm start</code>	Run start script
<code>npm run lint</code>	Run lint script
<code>npm run command</code>	Execute package binary
<code>git status</code>	Show Git status

<code>git add .</code>	Stage changes
<code>git commit -m "message"</code>	Create commit
<code>git push</code>	Push commits
<code>git pull</code>	Pull remote changes
<code>git clone URL</code>	Clone repository

43. The Most Important Mental Model

If you remember only one thing, remember this:

Node.js → Runs JavaScript outside browser

npm → Manages packages and scripts

TypeScript → Adds static types to JavaScript

React → Builds UI using components

Next.js → React framework with routing, server/client features, and build tooling

fetch() → Communicates with APIs

JSON → Common format for exchanging data

Supabase → Database/auth/backend services

Git → Tracks source-code changes

Build → Converts/prepares the application for deployment

Study order:

JavaScript → TypeScript → React → Hooks → API/HTTP → Next.js → Supabase → Git → Build/Deployment

React & Web Development Revision Notes

JavaScript • TypeScript • React • Next.js • npm • APIs • Git